LABORATORIO 5

Protocolo MQTT

Preparado por:
Luisdavid Colina V24766381
Samantha Ramírez V31307714

1. Configuración del servidor MQTT

Hasta este punto de la asignatura la comunicación entre equipos se había resuelto con sockets TCP, donde cada par de programas tenía que conocerse de antemano y mantener una conexión directa. MQTT cambia ese esquema por uno de publicación y suscripción intermediado por un broker, de modo que quien produce un dato y quien lo consume nunca se hablan de forma directa [1].

La guía plantea montar el servidor sobre una Raspberry Pi o, en su defecto, sobre una máquina virtual. En el laboratorio se trabajó directamente sobre los PC de las mesas, con Linux Mint de 64 bits, y el equipo que quedó como servidor fue el de dirección `190.169.74.68`.

1.1. Sin autenticación

Pasos en la máquina del servidor. Mosquitto está empaquetado en los repositorios oficiales de Ubuntu, así que la instalación se resolvió con `apt`. El paquete `mosquitto` arrastra tres dependencias: `libmosquitto1` con la implementación del protocolo, `libdlt2` para el registro de eventos y `libwebsockets19t64`, que es la que después permite exponer el broker también sobre WebSockets (Figura 1).

Código 1: Instalación del broker.

```
sudo apt install mosquitto -y
```

```
lacore4@lacore4-ThinkCentre-M92p:~$ sudo apt install mosquitto -y
[sudo] password for lacore4:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-hwe-6.14-tools-6.14.0-37 linux-image-6.14.0-37-generic
  linux-modules-6.14.0-37-generic linux-modules-extra-6.14.0-37-generic
  linux-tools-6.14.0-37-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libdlt2 libmosquitto1 libwebsockets19t64
The following NEW packages will be installed:
  libdlt2 libmosquitto1 libwebsockets19t64 mosquitto
0 upgraded, 4 newly installed, 0 to remove and 348 not upgraded.
Need to get 591 kB of archives.
After this operation, 1.650 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 libmosquitto1 amd64 2.0.18-1build3 [54,1 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 libdlt2 amd64 2.18.10-10 [65,4 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/universe amd64 libwebsockets19t64 amd64 4.3.3-1.1build3 [229 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/universe amd64 mosquitto amd64 2.0.18-1build3 [242 kB]
Fetched 591 kB in 3s (213 kB/s)
Selecting previously unselected package libmosquitto1:amd64.
(Reading database ... 587198 files and directories currently installed.)
Preparing to unpack .../libmosquitto1_2.0.18-1build3_amd64.deb ...
Unpacking libmosquitto1:amd64 (2.0.18-1build3) ...
Selecting previously unselected package libdlt2:amd64.
Preparing to unpack .../libdlt2_2.18.10-10_amd64.deb ...
Unpacking libdlt2:amd64 (2.18.10-10) ...
Selecting previously unselected package libwebsockets19t64:amd64.
Preparing to unpack .../libwebsockets19t64_4.3.3-1.1build3_amd64.deb ...
Unpacking libwebsockets19t64:amd64 (4.3.3-1.1build3) ...
Selecting previously unselected package mosquitto.
Preparing to unpack .../mosquitto_2.0.18-1build3_amd64.deb ...
Unpacking mosquitto (2.0.18-1build3) ...
Setting up libmosquitto1:amd64 (2.0.18-1build3) ...
Setting up libwebsockets19t64:amd64 (4.3.3-1.1build3) ...
Setting up libdlt2:amd64 (2.18.10-10) ...
Setting up mosquitto (2.0.18-1build3) ...
Created symlink /etc/systemd/system/multi-user.target.wants/mosquitto.service →
/usr/lib/systemd/system/mosquitto.service.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
```

Figura 1: Instalación del paquete `mosquitto` y sus dependencias.

Enseguida se editó el archivo de configuración para declarar puertos y protocolo. La instalación por defecto de Mosquitto 2.x solo acepta conexiones desde `localhost`, lo que en este laboratorio no sirve porque la idea era que otro grupo publicara contra nuestro broker. Por eso hubo que declarar de forma explícita un `listener` y permitir clientes anónimos. El archivo quedó como se muestra en el Código 2 y en la Figura 2.

Código 2: Líneas agregadas a `/etc/mosquitto/mosquitto.conf`.

```
sudo nano /etc/mosquitto/mosquitto.conf

listener 1883              # puerto por defecto
# habilitar websockets, opcional
listener 9001
protocol websockets
# permitir usuarios anonimos
allow_anonymous true
```

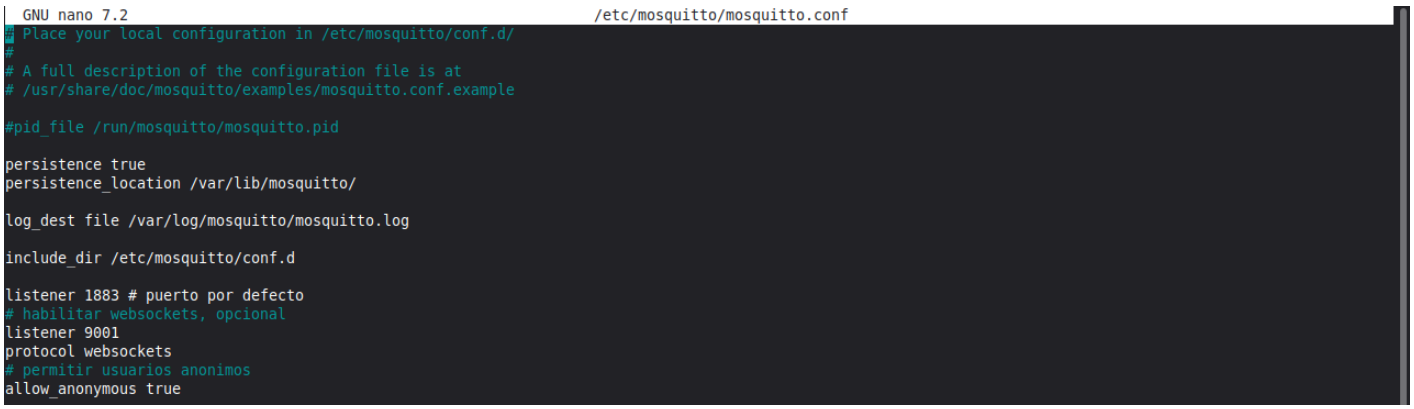


Figura 2: Archivo de configuración editado con `nano`.

Vale la pena detenerse en cada directiva. Con `persistence true`, que ya venía en el archivo, el broker vuelca a disco las suscripciones y los mensajes retenidos, de forma que un reinicio no borra el estado. `log_dest file` manda la bitácora a un archivo en lugar de dejarla solo en el diario de `systemd`, lo que facilita revisar conexiones fallidas. `include_dir` hace que todo lo que esté dentro de `/etc/mosquitto/conf.d` se cargue después del archivo principal, y esa carpeta es justamente la que se usó más adelante para la autenticación.

Los dos `listener` declarados atienden protocolos distintos sobre el mismo broker. El puerto 1883 es el puerto MQTT estándar sin cifrado [1] y es el que usan los clientes de consola. El puerto 9001 expone el mismo broker sobre WebSockets, que es la vía por la que un navegador podría suscribirse a un tema sin necesidad de un cliente MQTT nativo.

Antes de reiniciar el servicio se levantó el broker en primer plano contra esa configuración para comprobar que no tuviera errores de sintaxis, ya que si una directiva está mal escrita Mosquitto falla al arrancar y el servicio queda caído sin aviso claro.

Código 3: Verificación de la configuración y arranque del servicio.

```
sudo mosquitto -c /etc/mosquitto/mosquitto.conf

sudo systemctl start mosquitto
sudo systemctl enable mosquitto
sudo systemctl status mosquitto
```

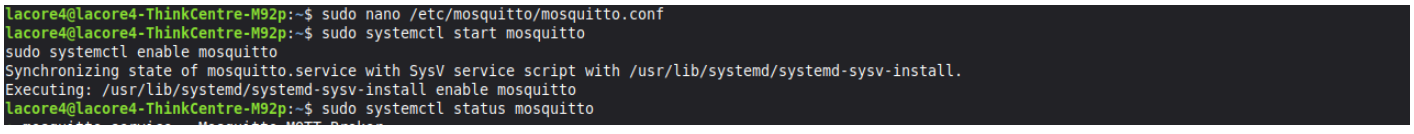


Figura 3: Arranque del servicio y sincronización con el script SysV.

El propio paquete crea el enlace simbólico en `multi-user.target.wants`, de manera que `systemd` ya reconoce la unidad apenas termina la instalación. Aun así se ejecutó `enable` de forma explícita para dejar constancia de que el broker debe levantarse solo en cada arranque, y no depender de que alguien lo inicie a mano.

La salida de `status` confirma el estado `active (running)` y deja ver algo útil para entender cómo se comporta el servicio: antes de levantar el proceso principal, `systemd` ejecuta cuatro pasos previos de `ExecStartPre` que crean `/var/log/mosquitto` y `/run/mosquitto` y les asignan el usuario `mosquitto`. El proceso corre entonces sin privilegios de administrador, leyendo su configuración desde `/etc/mosquitto/mosquitto.conf` (Figura 4).

```
● mosquitto.service - Mosquitto MQTT Broker
     Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-09-11 11:17:07 -04; 2min 53s ago
       Docs: man:mosquitto.conf(5)
             man:mosquitto(8)
    Process: 17237 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0/SUCCESS)
    Process: 17239 ExecStartPre=/bin/chown mosquitto:mosquitto /var/log/mosquitto (code=exited, status=0/SUCCESS)
    Process: 17241 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SUCCESS)
    Process: 17243 ExecStartPre=/bin/chown mosquitto:mosquitto /run/mosquitto (code=exited, status=0/SUCCESS)
   Main PID: 17245 (mosquitto)
      Tasks: 1 (limit: 18601)
     Memory: 2.2M (peak: 2.9M)
        CPU: 131ms
     CGroup: /system.slice/mosquitto.service
             └─17245 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

Sep 11 11:17:07 lacore4-ThinkCentre-M92p systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...
Sep 11 11:17:07 lacore4-ThinkCentre-M92p systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.
```

Figura 4: Estado del servicio `mosquitto.service` tras el arranque.

Pasos en la máquina cliente. El paquete `mosquitto` solo trae el broker. Las herramientas de línea de comandos `mosquitto_pub` y `mosquitto_sub` vienen aparte, en `mosquitto-clients`, y se instalaron en los equipos que participaron en la prueba (Figura 5).

Código 4: Instalación de los clientes de consola.

```
sudo apt install mosquitto-clients -y
```

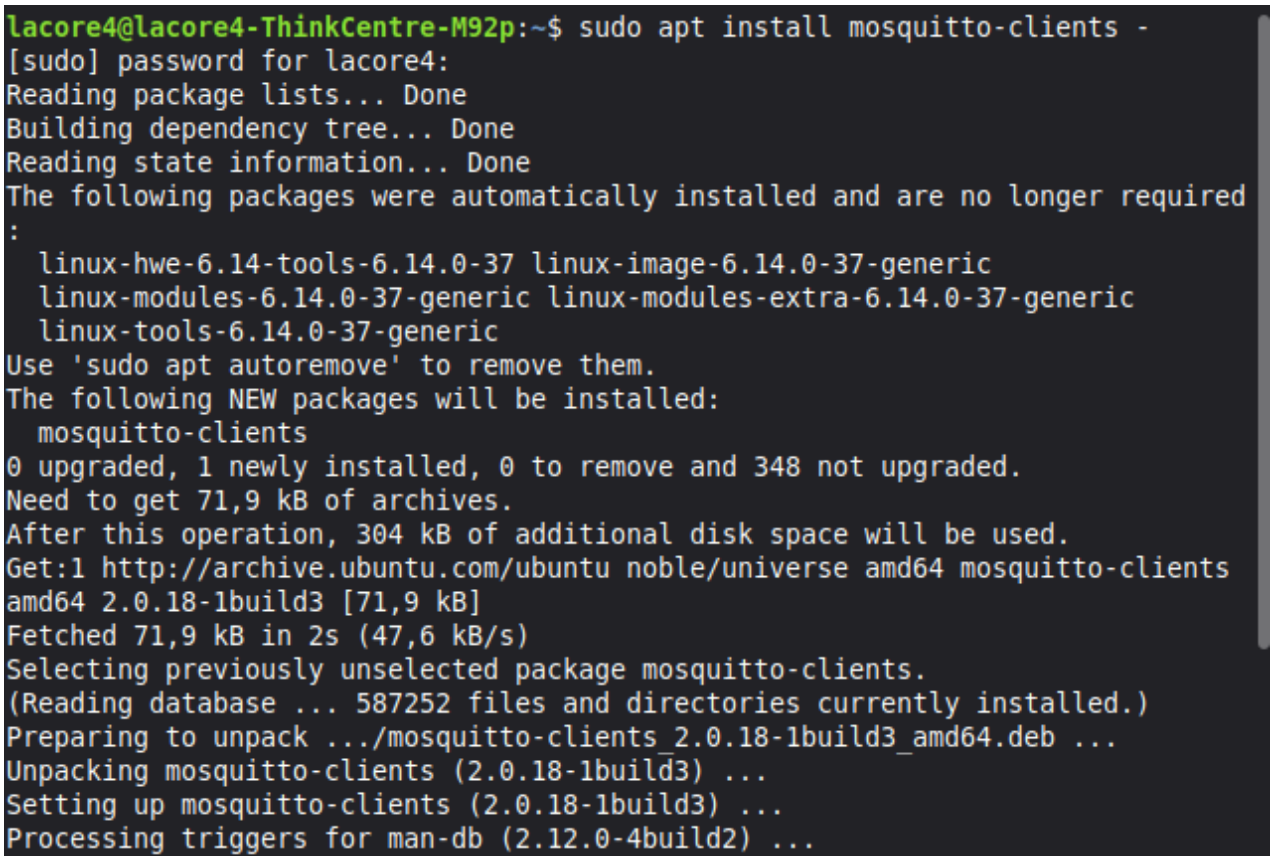


Figura 5: Instalación del paquete `mosquitto-clients`.

Antes de intentar cualquier publicación se comprobó que hubiera camino de red hasta el equipo del broker. Esta verificación evita confundir dos problemas muy distintos: que el broker no esté escuchando, o que el equipo ni siquiera sea alcanzable. Los cinco paquetes ICMP llegaron sin pérdidas y con tiempos por debajo del milisegundo, lo que descarta el segundo caso (Figura 6).

```
lacore4@lacore4-ThinkCentre-M92p:~$ ping  190.169.74.68
PING 190.169.74.68 (190.169.74.68) 56(84) bytes of data.
64 bytes from 190.169.74.68: icmp_seq=1 ttl=64 time=0.042 ms
64 bytes from 190.169.74.68: icmp_seq=2 ttl=64 time=0.039 ms
64 bytes from 190.169.74.68: icmp_seq=3 ttl=64 time=0.048 ms
64 bytes from 190.169.74.68: icmp_seq=4 ttl=64 time=0.052 ms
64 bytes from 190.169.74.68: icmp_seq=5 ttl=64 time=0.062 ms
^C
--- 190.169.74.68 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4098ms
rtt min/avg/max/mdev = 0.039/0.048/0.062/0.008 ms
```

Figura 6: Prueba de alcance hacia el equipo que aloja el broker.

Poniendo a prueba la instalación. Con el broker escuchando en el puerto 1883 y aceptando clientes anónimos, la prueba consistió en dejar una terminal suscrita a un tema y publicar en él desde otra. Se eligió el tema `kjbhik`, un nombre arbitrario, precisamente para asegurar que ningún otro grupo estuviera publicando en el mismo espacio de nombres durante la práctica.

Código 5: Suscripción y publicación sobre el mismo tema.

```
# Terminal suscriptora
mosquitto_sub -h 190.169.74.68 -t kjbhik

# Terminal publicadora
mosquitto_pub -h 190.169.74.68 -t kjbhik -m "Prueba exitosa"
```

El comportamiento observado deja clara la diferencia con el esquema de sockets del laboratorio anterior. La terminal suscriptora queda bloqueada esperando, sin conocer quién va a publicar ni cuántos publicadores hay. La terminal publicadora, por su parte, envía el mensaje y termina de inmediato, sin saber si alguien estaba escuchando. Cada una de las cinco ejecuciones de `mosquitto_pub` produjo exactamente una línea nueva en el suscriptor (Figura 7).

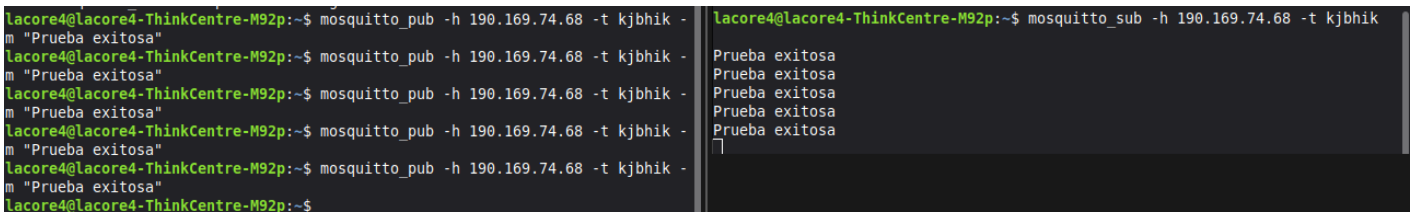


Figura 7: Publicador (izquierda) y suscriptor (derecha) sobre el tema `kjbhik`.

Un detalle que conviene anotar es que los mensajes publicados antes de que el suscriptor se conectara no aparecen. Sin la bandera `-r` el broker no retiene el último valor de un tema, de modo que solo se entrega lo que se publica mientras la suscripción está activa.

Pendiente de ejecución en el laboratorio. Falta repetir el intercambio con las etiquetas `cliente1`, `cliente2` y `cliente3` entre las tres máquinas del laboratorio, de forma que cada una reciba los mensajes de las otras dos.

1.2. Autenticación por usuario y clave

Dejar `allow_anonymous true` significa que cualquier equipo de la red puede leer o escribir en cualquier tema, algo inaceptable en un despliegue real de IoT. El siguiente paso fue entonces crear un archivo de credenciales con `mosquitto_passwd`. La bandera `-c` crea el archivo desde cero, por lo que solo debe usarse la primera vez: volver a invocarla borra los usuarios que ya existían. El alias de usuario elegido fue `admin`.

Código 6: Creación del usuario, permisos y reinicio del broker.

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd "admin"
sudo chmod 755 /etc/mosquitto/passwd
sudo systemctl restart mosquitto
```

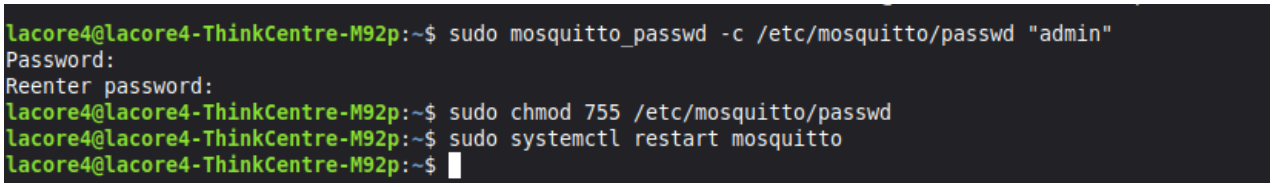


Figura 8: Creación del usuario `admin` y reinicio del servicio.

El ajuste de permisos con `chmod` fue necesario porque el proceso del broker no corre como administrador, sino con el usuario `mosquitto`, y sin permiso de lectura sobre el archivo de contraseñas rechaza todas las conexiones. Hecho esto, se agregaron las dos directivas de autenticación dentro de `/etc/mosquitto/conf.d` (Figura 9). Estas líneas se cargan después del archivo principal gracias al `include_dir` configurado antes, de modo que sobrescriben el `allow_anonymous true` original.

Código 7: Directivas de autenticación.

```
allow_anonymous false
password_file /etc/mosquitto/passwd
```

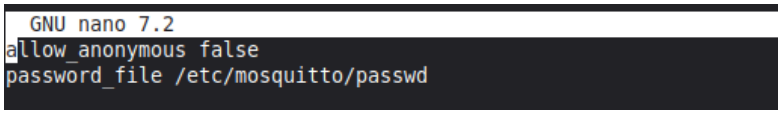


Figura 9: Directivas de autenticación cargadas desde `conf.d`.

Poniendo a prueba la instalación. Tras el reinicio, los mismos comandos de la sección anterior dejaron de funcionar sin credenciales. Con las banderas `-u` y `-P` la comunicación se restableció y el suscriptor volvió a recibir cada mensaje publicado (Figura 10).

Código 8: Suscripción y publicación autenticadas.

```
# Terminal suscriptora
sudo mosquitto_sub -h 190.169.74.68 -t test -u "admin" -P "12345"

# Terminal publicadora
sudo mosquitto_pub -h 190.169.74.68 -t test -m "mensaje prueba" -u "admin" -P "12345"
```

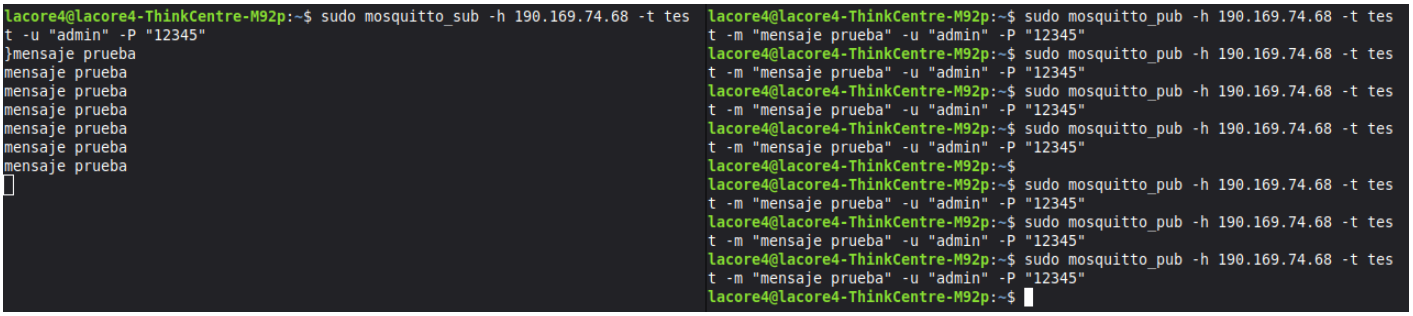


Figura 10: Intercambio autenticado sobre el tema `test`.

Conviene señalar que esta autenticación viaja en claro sobre el puerto 1883. Cualquiera con acceso al segmento de red puede capturar el usuario y la contraseña con un analizador de tráfico. En un despliegue real el paso siguiente sería habilitar TLS en el puerto 8883, que es el puerto reservado para MQTT cifrado [1].

1.3. Publicación y suscripción por código

Pendiente de ejecución en el laboratorio. Falta ejecutar el cliente de ejemplo en Python con la librería `paho-mqtt`, apuntando al broker configurado, y dejar constancia de la salida de los callbacks `on_connect` y `on_message`.

2. Desarrollo de actividades

2.1. Actividad 3.1: Conexión del sensor de ultrasonido

Enunciado: Conecte un sensor de ultrasonido a la placa Raspberry Pi, muestre el circuito en Fritzing y agregue una foto de su prototipo. (5 puntos)

Pendiente de ejecución en el laboratorio. Falta el montaje sobre la Raspberry Pi, el diagrama en Fritzing y la fotografía del prototipo.

2.2. Actividad 3.2: Lectura periódica del sensor

Enunciado: Modifique el código mostrado para que cada 10 segundos se realice una lectura del sensor de ultrasonido y se muestre la salida en la terminal. (5 puntos)

Pendiente de ejecución en el laboratorio. Falta adaptar el intervalo del lazo de medición a 10 segundos y capturar la salida en la terminal de la Raspberry Pi.

2.3. Actividad 3.3: Publicación de las lecturas al broker propio

Enunciado: Modifique el código para que las lecturas del sensor sean publicadas al broker MQTT con el tópico `/GRUPO/SENSOR`. Muestre la salida de la terminal del broker. (5 puntos)

Pendiente de ejecución en el laboratorio. Falta unir la lectura del sensor con el cliente `paho-mqtt` y capturar la salida del broker recibiendo el tópico.

2.4. Actividad 3.4: Publicación al broker de otro grupo

Enunciado: Modifique el código anterior para publicar el tópico al broker de otro de los grupos de trabajo. (5 puntos)

Pendiente de ejecución en el laboratorio. Falta coordinar con otro grupo la dirección de su broker y sus credenciales, y dejar constancia de la recepción del lado de ellos.

3. Conclusiones

En este laboratorio se pasó de un modelo de comunicación punto a punto a uno mediado por un broker. La diferencia práctica quedó clara en la prueba de la sección 1.1: el publicador no necesita saber cuántos suscriptores hay ni si alguno está conectado, y el suscriptor no necesita saber de dónde viene el dato. Ese desacoplamiento es lo que hace viable una red con muchos sensores y varios consumidores del mismo dato.

La configuración del broker mostró que Mosquitto 2.x es restrictivo por defecto y solo escucha en `localhost`. Declarar el `listener` de forma explícita no es un detalle opcional, es lo que permite que el broker sirva a otros equipos de la red.

El mecanismo de `include_dir` resultó ser la forma limpia de manejar la configuración. Mantener la autenticación en un archivo aparte dentro de `conf.d`, en lugar de editar el archivo principal, permitió activar y desactivar ese comportamiento sin tocar el resto de las directivas.

Finalmente, el ajuste de permisos sobre el archivo de contraseñas dejó ver que el broker corre sin privilegios de administrador. Es un detalle fácil de pasar por alto que se manifiesta como un rechazo generalizado de conexiones, sin un mensaje que apunte directo a la causa.

Referencias

[1] A. Banks, E. Briggs, K. Borgendale y R. Gupta, "MQTT Version 5.0," *OASIS Standard*, mar. 2019. [En línea]. Disponible: `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html`. [Consulta: sep. 2026]

[2] Eclipse Foundation, "mosquitto.conf, the configuration file for mosquitto," *Eclipse Mosquitto Documentation*, 2024. [En línea]. Disponible: `https://mosquitto.org/man/mosquitto-conf-5.html`. [Consulta: sep. 2026]

[3] Eclipse Foundation, "mosquitto_passwd, password file generation utility for mosquitto," *Eclipse Mosquitto Documentation*, 2024. [En línea]. Disponible: `https://mosquitto.org/man/mosquitto_passwd-1.html`. [Consulta: sep. 2026]

[4] Eclipse Foundation, "Eclipse Paho MQTT Python Client," 2024. [En línea]. Disponible: `https://eclipse.dev/paho/index.php?page=clients/python/index.php`. [Consulta: sep. 2026]

[5] A. Russoniello, "Laboratorio: Protocolo MQTT," guía de práctica de laboratorio, Laboratorio General (Internet de las Cosas), Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela, Caracas, 2026.